

Clase 5: Regularizacion, Grid Search y SVM

Diego Stalder
dstalder@ing.una.py

Facultad de Ingenieria
Universidad Nacional de Asuncion

12 de Agosto de 2026

1. Regularizacion

- L1 (Lasso), L2 (Ridge), Elastic Net
- Ejemplo simple con 2 variables

2. Grid Search

- Optimizacion de hiperparametros
- Cross-validation

3. SVM - Support Vector Machines

- Caso lineal: margen maximo
- Kernel trick: de 2D a altas dimensiones
- Ecuaciones clave

4. PAUSAS PRACTICAS

- Implementacion en Jupyter
- Prediccion: pasa o no pasa

Objetivo

Predecir aprobacion

usando SVM con
el dataset academico

SVM: de lo simple a lo poderoso

Regularizacion: ¿Por que es necesaria?

Problema

Overfitting en modelos lineales

Modelo sin regularizacion:

- Coeficientes grandes
- Sobreajuste a ruido
- Poca generalizacion

Ejemplo:

$$y = 1000x_1 - 999x_2 + 0,5$$

Coeficientes muy grandes → inestable

Modelo con regularizacion:

- Penaliza coeficientes grandes
- Mejor generalizacion
- Mas robusto

Ejemplo:

$$y = 0,5x_1 + 0,3x_2 + 0,5$$

Coeficientes pequenos → estable

La regularizacion controla la complejidad del modelo

Regularizacion L2: Ridge

Funcion de Costo con L2

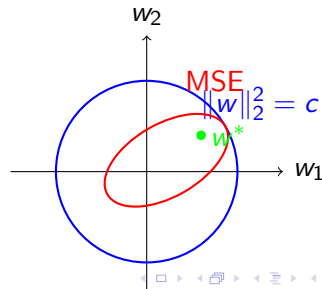
$$J(\mathbf{w}) = \underbrace{\frac{1}{2n} \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2}_{\text{MSE}} + \underbrace{\alpha \sum_{j=1}^p w_j^2}_{\text{Penalizacion L2}}$$

Ejemplo 2D:

$$J(w_1, w_2) = \frac{1}{2n} \sum (y_i - w_1 x_{i1} - w_2 x_{i2})^2 + \alpha (w_1^2 + w_2^2)$$

Efecto:

- Encoge coeficientes
- Ninguno llega a 0
- Solucion unica



Regularizacion L1: Lasso

Funcion de Costo con L1

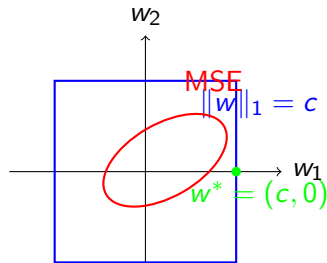
$$J(\mathbf{w}) = \underbrace{\frac{1}{2n} \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2}_{\text{MSE}} + \underbrace{\alpha \sum_{j=1}^p |w_j|}_{\text{Penalizacion L1}}$$

Ejemplo 2D:

$$J(w_1, w_2) = \frac{1}{2n} \sum (y_i - w_1 x_{i1} - w_2 x_{i2})^2 + \alpha(|w_1| + |w_2|)$$

Efecto:

- Puede llevar a 0
- **Selección de features**
- Solucion no unica (en esquinas)



Elastic Net: Lo mejor de ambos

Funcion de Costo con Elastic Net

$$J(\mathbf{w}) = \underbrace{\frac{1}{2n} \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2}_{\text{MSE}} + \underbrace{\alpha \rho \sum_{j=1}^p |w_j|}_{\text{L1}} + \underbrace{\frac{\alpha(1-\rho)}{2} \sum_{j=1}^p w_j^2}_{\text{L2}}$$

Hiperparametros:

- α : fuerza de regularizacion
- ρ : balance L1/L2
 - $\rho = 1$: Lasso
 - $\rho = 0$: Ridge
 - $0 < \rho < 1$: mezcla

Ventajas:

- Selecciona features (L1)
- Agrupa features correlacionados (L2)
- Mas flexible que L1 o L2 solos

Elastic Net es el mejor de ambos mundos

PAUSA PRACTICA - A CODIFICAR

Implementar Ridge, Lasso y Elastic Net con sklearn

Tareas

1. Cargar datos academicos (X, y)
2. Dividir en train/test
3. Entrenar:
 - `Ridge( $\alpha$ )`
 - `Lasso( $\alpha$ )`
 - `ElasticNet( $\alpha$ , l1_ratio)`
4. Comparar coeficientes
5. Evaluar en test

Preguntas para reflexionar

- ¿Que pasa con los coeficientes cuando α aumenta?
- ¿Cual da mejor accuracy en test?
- ¿Que features se eliminan con Lasso?

Grid Search: Búsqueda de Hiperparametros

Problema

¿Como elegir los mejores hiperparametros?

Estrategia de Grid Search:

1. Definir grilla de valores
2. Probar todas las combinaciones
3. Evaluar con cross-validation
4. Seleccionar la mejor

Ejemplo:

- $\alpha = [0.001, 0.01, 0.1, 1]$
- $l1_ratio = [0, 0.5, 1]$
- $4 \times 3 = 12$ combinaciones

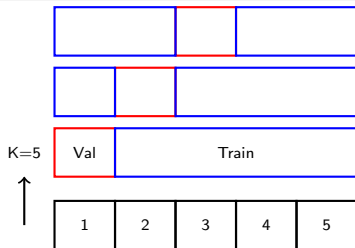
GridSearchCV en sklearn

```
param_grid =  
'alpha': [0.001, 0.01, 0.1, 1], 'l1_ratio': [0, 0.5, 1]  
grid = GridSearchCV(ElasticNet(), param_grid, cv = 5)  
grid.fit(X_train, y_train)  
print(grid.best_params_)
```


Cross-Validation en Grid Search

Validacion Cruzada (K-Fold)

1. Dividir datos en K partes (folds)
2. Para cada fold:
 - Entrenar en K-1 folds
 - Validar en el fold restante
3. Promediar metricas



PAUSA PRACTICA - A CODIFICAR

Optimizar hiperparametros con GridSearchCV

Tareas

1. Definir grilla para:
 - Ridge: alpha
 - Lasso: alpha
 - ElasticNet: alpha, l1_ratio
2. Aplicar GridSearchCV con K=5
3. Comparar mejores modelos
4. Evaluar en test
5. Visualizar resultados

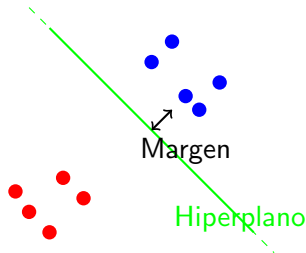
Preguntas

- ¿Que combinacion da mejor accuracy?
- ¿Cuanto tiempo tarda la busqueda?
- ¿Se puede hacer Random Search en vez de Grid?

SVM: Support Vector Machines

Idea Principal

Encontrar el **hiperplano** que separe las clases con el **margen maximo**



Problema Primal

Dados puntos $(\mathbf{x}_i, y_i)$ con $y_i \in \{-1, 1\}$, encontrar:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

sujeto a:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i$$

Interpretacion:

- $\mathbf{w}$: vector normal al hiperplano
- b : sesgo
- $\frac{1}{\|\mathbf{w}\|}$: margen
- Minimizar $\|\mathbf{w}\| \rightarrow$ maximizar margen

Hiperplano:

$$\mathbf{w}^T \mathbf{x} + b = 0$$

Clasificacion:

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$$

SVM con Margen Suave (C-SVM)

Problema con variables de slack

$$\min_{\mathbf{w}, b, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

sujeto a:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \forall i$$

Hiperparametro C:

- **C grande:** margen estricto (poco error)
- **C pequeno:** margen suave (mas error)
- Controla el trade-off

Interpretacion:

- ξ_i : violacion del margen
- $C \sum \xi_i$: penalizacion
- Permite datos no separables

C es el principal hiperparametro en SVM

Problema Dual

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

sujeto a:

$$0 \leq \alpha_i \leq C, \quad \sum_{i=1}^n \alpha_i y_i = 0$$

Puntos Clave:

- α_i : multiplicadores de Lagrange
- Solo $\alpha_i > 0$ son **vectores soporte**
- $\mathbf{w} = \sum \alpha_i y_i \mathbf{x}_i$

Kernel Trick:

$$\mathbf{x}_i^T \mathbf{x}_j \rightarrow K(\mathbf{x}_i, \mathbf{x}_j)$$

- Mapeo a altas dimensiones
- Sin calcular explícitamente

Kernel Trick: De 2D a 3D

Problema: Datos no separables en 2D



Mapeo ϕ :

$$\phi(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2)$$

- Va de 2D a 3D
- Ahora son separables!

Kernel correspondiente:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

- Se calcula eficientemente
- Ejemplo: $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + 1)^2$

Kernel	Formula	Cuando usar
Lineal	$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$	Datos linealmente separables
Polinomial	$K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + r)^d$	Datos con interacciones polinomiales
RBF (Gaussiano)	$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \ \mathbf{x}_i - \mathbf{x}_j\ ^2)$	Mas usado, funciona en general
Sigmoid	$K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\kappa \mathbf{x}_i^T \mathbf{x}_j + c)$	Redes neuronales

Hiperparametros del RBF

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

- γ (gamma): Controla el alcance del kernel
- γ grande $\rightarrow$ decision local
- γ pequeno $\rightarrow$ decision global
- C y γ son los 2 hiperparametros clave

SVM: Ejemplo Completo

Clasificación con SVM (sklearn)

```
Cargar datos academicos X = df[['Primer.Par', 'Segundo.Par', 'Convocatoria']] y = df['Aprobado'] 0 o 1
Preprocesar scaler = StandardScaler() X_scaled = scaler.fit_transform(X)
Dividir X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size = 0.2)
Entrenar SVM con RBF svm = SVC(kernel='rbf', C=1.0, gamma='scale') svm.fit(X_train, y_train)
Evaluar accuracy = svm.score(X_test, y_test) print(f'Accuracy: accuracy:.4f')
```

PAUSA PRACTICA - A CODIFICAR

Implementar SVM para predecir "pasa o no pasa"

Tareas

1. Usar features simples:
 - Primer.Par, Segundo.Par
 - Convocatoria, TPLab., Lab.
2. Entrenar:
 - `SVC(kernel='linear')`
 - `SVC(kernel='rbf', C=1, gamma='scale')`
3. Comparar accuracy
4. Visualizar margenes (con 2D)

Preguntas

- ?Cual kernel da mejor accuracy?
- ?Que pasa si cambias C y gamma?
- ?Como se comparan con Regresion Logistica?

Busqueda de hiperparametros para SVM

```
Definir grilla paramgrid =  
'C': [0.1, 1, 10, 100], 'gamma': [0.001, 0.01, 0.1, 1], 'kernel': ['rbf']  
GridSearch grid = GridSearchCV(SVC(), paramgrid, cv = 5, scoring = 'accuracy')gridfit( $X_{train}$ ,  $y_{train}$ )  
print(f'Mejores params: grid.bestparams')print(f'Mejorscore : gridbestscore.4f')print(f'Testscore : gridscore( $X_{test}$ ,  $y_{test}$ ) : 4f')
```

Grid Search encuentra los mejores C y gamma

Comparacion: Regresion Logistica vs SVM

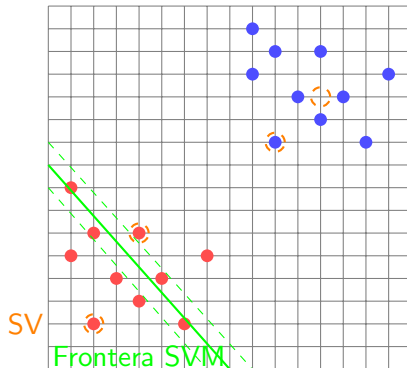
Modelo	Train	Test	Diferencia
Regresion Logistica	86.2 %	85.8 %	0.4 %
SVM Lineal	86.5 %	86.1 %	0.4 %
SVM RBF (C=1, $\gamma=0.1$)	88.7 %	87.3 %	1.4 %
SVM RBF (optimizado)	89.1 %	87.6 %	1.5 %

Observaciones

- SVM RBF supera a Regresion Logistica
- C y gamma afectan significativamente
- Grid Search es necesario para optimizar
- SVM es mas flexible que RL

SVM es un modelo poderoso para datos tabulares

Visualizacion: SVM en 2D



Los vectores soporte son los mas importantes

PAUSA PRACTICA - A CODIFICAR

Optimizar SVM con Grid Search y evaluar

Tareas

1. Usar GridSearchCV para SVM RBF
2. Probar:
 - C: [0.1, 1, 10, 100]
 - gamma: [0.001, 0.01, 0.1, 1]
3. Usar cross-validation (K=5)
4. Comparar con el mejor modelo lineal
5. Analizar vectores soporte

Preguntas finales

- ¿Mejoro la prediccion de aprobacion?
- ¿Que combinacion C-gamma es mejor?
- ¿Se puede usar SVM para regresion (SVR)?

Conclusiones

Hoy aprendimos

- **Regularizacion:** L1, L2 y Elastic Net
- **Grid Search:** Optimizacion de hiperparametros
- **SVM:** Desde el margen maximo al kernel trick
- Implementamos en **Jupyter** con datos reales
- Predecimos **aprobacion** con SVM

Proximos pasos

- Probar SVM con features del estudio CNMAC
- Comparar con MLP
- Implementar SVR para regresion
- Aplicar a prediccion de notas finales

SVM: un clasificador poderoso para problemas reales

?Preguntas? Sobre Regularizacion, Grid Search y SVM

¡A seguir practicando en el notebook!